

How the Learning Loop Works

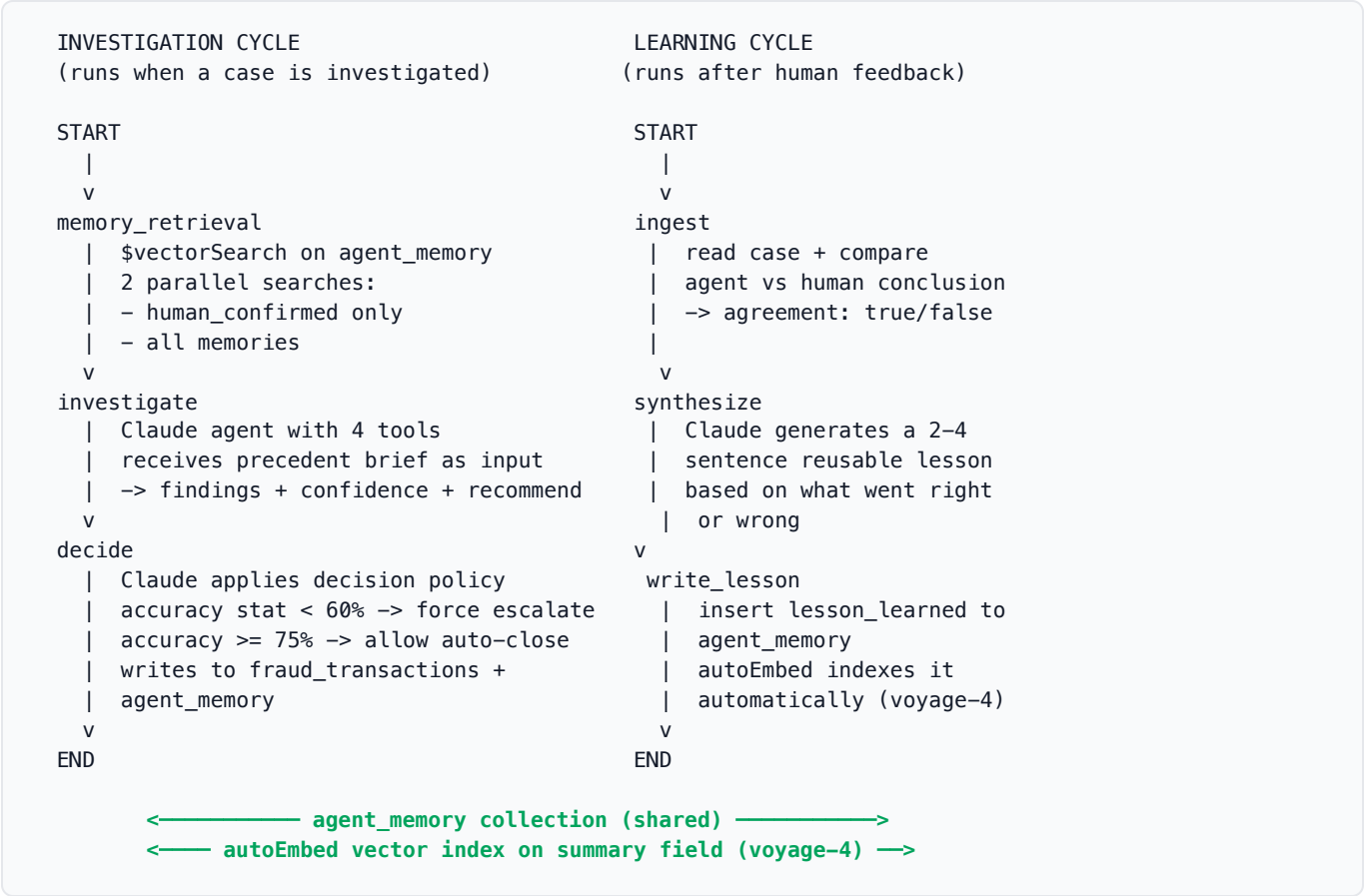
Memory-Curated Investigation & Feedback Cycle

Fraud Detection & Multi-Agent System — Powered by MongoDB Atlas

September 2026

Overview

The system has a **closed-loop learning cycle**: every human review of an AI investigation produces a searchable lesson that improves future investigations. The loop has two graphs that share a single MongoDB collection (`agent_memory`) as their memory substrate.



The Agents

Agent	Graph	LLM?	Role in the learning loop
Memory Retrieval	Investigation	No	Reads <code>agent_memory</code> via <code>\$vectorSearch</code> . Finds past lessons and computes the accuracy stat. Builds the precedent brief that the investigator reads.
Investigation	Investigation	Yes	Claude agent. Receives the precedent brief as input. Investigates the case with 4 tools. Produces findings, confidence, and a recommendation.
Decision	Investigation	Yes	Claude agent. Applies a decision policy that uses the accuracy stat. Writes findings to <code>fraud_transactions</code> and an <code>investigation_summary</code> to <code>agent_memory</code> .

Outcome Ingest	Learning	No	Reads the investigated case. Compares what the agent concluded vs. what the human decided. Produces an <code>agreement</code> boolean.
Memory Synthesis	Learning	Yes	Claude agent. Generates a concise, reusable lesson from the case outcome. Agreement → "confirmed pattern" lesson. Disagreement → "correction" lesson.
Memory Write	Learning	No	Builds a <code>lesson_learned</code> document and inserts it into <code>agent_memory</code> . Atlas autoEmbed indexes it automatically.

The `agent_memory` Collection

All memory lives in a single MongoDB collection with an autoEmbed vector index on the `summary` field (model: `voyage-4`). Three document types coexist:

Document type 1: `investigation_summary`

Written by the Decision Agent after every investigation. This is the agent's own unverified conclusion.

```
{
  "type": "investigation_summary",
  "transaction_id": "txn_A",
  "summary": "Flagged as fraud: high amount ($4,800) from Nigeria on new device...",
  "source": "agent",
  "outcome": "escalated",
  "tags": ["high_amount", "new_device"]
}
```

source: agent Treat as prior reasoning, not fact. A human has not verified this yet.

Document type 2: `analyst_feedback`

Written by the feedback endpoint when a human submits a verdict. This is the raw audit trail.

```
{
  "type": "analyst_feedback",
  "transaction_id": "txn_A",
  "summary": "Human analyst reviewed transaction txn_A and confirmed outcome: false_positive...",
  "source": "human_confirmed",
  "outcome": "false_positive",
  "reviewed_by": "analyst@email.com"
}
```

source: human_confirmed Ground truth — a human verified this outcome.

Document type 3: `lesson_learned` key output of the learning graph

Written by the Learning Graph after synthesizing the human's feedback. This is the reusable lesson.

```
{
  "type": "lesson_learned",
  "transaction_id": "txn_A",
  "summary": "For transactions with [new_device] where the user has 3+ years of clean history and the device is the same OS as their previous device, the agent flagged this but it was a false positive. Key
```

```
        insight: check if the new device is an upgrade before flagging.",
"source": "human_confirmed",
"lesson_type": "false_positive_lesson",
"agreement": false,
"indicators": ["high_amount", "new_device"],
"outcome": "false_positive",
"agent_conclusion": "flagged_as_fraud",
"human_conclusion": "false_positive"
}
```

source: human_confirmed

agreement: false

The agent was wrong. The lesson explains why so future investigations avoid the same mistake.

The Trust Gradient

The `source` field creates a trust gradient. The Memory Retrieval Agent runs two parallel vector searches: one filtered to `source: "human_confirmed"` (ground truth), one unfiltered (all precedent including unverified agent conclusions). The system prompt coaches the investigation agent: "Prefer results with `source: "human_confirmed"` as ground truth; treat `source: "agent"` results as unverified prior reasoning."

Walkthrough: Three Investigations

Investigation #1 — no lessons yet

A new case `txn_A` is flagged with indicators `[high_amount, new_device]`.

1. **Memory Retrieval Agent** runs `$vectorSearch` on `agent_memory`. No `lesson_learned` documents exist → the precedent brief is empty.
2. **Investigation Agent** investigates with no prior context. All indicators fire → `confidence: "high"`, `recommendation: "escalate"`.
3. **Decision Agent** writes findings to `fraud_transactions.investigation` and an `investigation_summary` to `agent_memory` with `source: "agent"`.
4. **Human** clicks "False positive" in the UI.
5. **Learning Graph** runs:
 - `ingestOutcome` reads the case: agent said fraud, human said false_positive → `agreement: false`
 - `synthesizeLesson` (Claude) generates: *"For transactions with [new_device] where the user has 3+ years of clean history and the device is the same OS, the agent flagged this but it was a false positive. Check if the new device is an upgrade before flagging."*
 - `write_lesson` inserts a `lesson_learned` document to `agent_memory`. `autoEmbed` indexes it automatically.

After Investigation #1

`agent_memory` now has 1 `lesson_learned` doc (false_positive_lesson, agreement=false).

Accuracy stat for `[high_amount, new_device]`: **0% (0 of 1 agreed)**

Investigation #2 — 1 lesson in memory

A similar case `txn_B` is flagged with the same indicators `[high_amount, new_device]`.

1. **Memory Retrieval Agent** runs `$vectorSearch`. Now finds the lesson from Investigation #1:

- **Verified search** (filtered to `human_confirmed`): finds the `lesson_learned` doc → places it under "CONFIRMED FALSE-POSITIVE PRECEDENTS"
- **Accuracy stat:** `$match + $group` on `lesson_learned` docs with `indicators: { $all: ["high_amount", "new_device"] }` → returns `{ total: 1, agreed: 0, rate: 0 }`

2. **Investigation Agent** receives this brief:

```
CONFIRMED FALSE-POSITIVE PRECEDENTS (human-verified):
1. txn_A – For transactions with [new_device] where the user has
  3+ years of clean history and the device is the same OS, the agent
  flagged this but it was a false positive. Check if the new device
  is an upgrade before flagging.

ACCURACY STAT: 0% (0 of 1 agreed)
```

The agent now checks: is this device an upgrade of the same OS? If yes, it might lower its confidence. If the location is unusual (e.g., Nigeria) and the user has no travel history, it might still flag → `confidence: "high"`, `recommendation: "escalate"`.

3. **Decision Agent** sees accuracy = 0% (< 60%) → **forces human escalation** regardless of confidence.

4. **Human** clicks "Confirm fraud" — this one was real fraud.

5. **Learning Graph** runs: `agreement: true` → writes a `confirmed_fraud_lesson`.

After Investigation #2

`agent_memory` now has 2 `lesson_learned` docs:

- `false_positive_lesson` from `txn_A` (agreement=false)
- `confirmed_fraud_lesson` from `txn_B` (agreement=true)

Accuracy stat for `[high_amount, new_device]`: **50% (1 of 2 agreed)**

Investigation #3 — 2 lessons in memory

Another case `txn_C` with the same indicators.

1. **Memory Retrieval Agent** runs `$vectorSearch`. Both lessons match:

- **Confirmed fraud precedents:** `txn_B`'s lesson ("new device + Nigeria IP + no travel history = account takeover")
- **Confirmed false-positive precedents:** `txn_A`'s lesson ("same OS upgrade + long history = false positive")
- **Accuracy stat:** 50% (1 of 2 agreed)

2. **Investigation Agent** now has **both sides of the story**:

```
CONFIRMED FRAUD PRECEDENTS (human-verified):
1. txn_B - new device + high amount + Nigeria IP with no travel
   history = confirmed account takeover fraud.

CONFIRMED FALSE-POSITIVE PRECEDENTS (human-verified):
1. txn_A - new device where user has 3+ years of clean history
   and device is same OS = false positive. Check if device is an
   upgrade before flagging.

ACCURACY STAT: 50% (1 of 2 agreed)
```

The agent can now distinguish between the two sub-patterns:

- Same OS + long history → likely false positive (lesson from `txn_A`)
- Different OS + unusual location + no travel history → likely fraud (lesson from `txn_B`)

3. **Decision Agent** sees accuracy = 50% (< 60%) → still forces human escalation. The system knows it's unreliable on this pattern.

After Investigation #3 (before feedback)

The investigation agent has a **decision tree** it wouldn't have without the lessons. It can tell the human analyst:

"This case resembles `txn_B` (confirmed fraud) more than `txn_A` (false positive) because the location is unusual and the device is a different OS. However, my accuracy on this pattern is only 50%, so I'm escalating for human review."

How the Accuracy Stat Scales

The accuracy stat is computed per indicator combination. The aggregation matches `lesson_learned` documents where `indicators` includes all the current case's indicators, then counts how many had `agreement: true`.

```
// computeAccuracyStat() in memory-retrieval-agent.js
{ $match: { type: "lesson_learned", source: "human_confirmed" } },
{ $match: { indicators: { $all: ["high_amount", "new_device"] } } },
{ $group: { _id: null, total: { $sum: 1 }, agreed: { $sum: { $cond: ["$agreement", 1, 0] } } } }
```

After N feedbacks	Agreed	Accuracy	Decision Agent behavior
1	0	0%	Force escalation (< 60%)
2	1	50%	Force escalation (< 60%)
3	2	67%	Follow recommendation (60–75%)
5	4	80%	Allow auto-close if confidence is high ($\geq 75\%$)
10	9	90%	Allow auto-close if confidence is high ($\geq 75\%$)

Each indicator combination has its own independent track record. `[high_amount, new_device]` might be at 80% while `[unusual_location, new_device]` is at 30% — the system learns separately for each fraud pattern type.

The Decision Agent's Policy

The Decision Agent receives the accuracy stat in its context and applies these rules:

Decision Rules

- **Accuracy < 60%** → always escalate to human, regardless of confidence or recommendation. The agent is frequently wrong on this pattern.
- **Accuracy $\geq 75\%$ AND confidence = "high" AND recommendation = "auto_close"** → auto-close the case. The agent is reliable on this pattern.
- **Confidence = "medium" or "low"** → always escalate, regardless of accuracy.
- **Confidence = "high" AND recommendation = "escalate"** → escalate to human.
- **No accuracy stat** (insufficient history) → follow the recommendation but lean toward escalation.

This means the system **adjusts its own autonomy threshold** based on past performance. Patterns where the agent is frequently wrong get forced to human review. Patterns where it's consistently right can be auto-closed without human intervention.

MongoDB Integration

autoEmbed — no client-side embedding

The `agent_memory` collection has an autoEmbed vector index on the `summary` field using the `voyage-4` model. Atlas generates and stores embeddings at write time. At query time, `$vectorSearch` accepts plain text — no Voyage API key in the application, no client-side embedding call.

```
// Index definition (indexes/agent_memory_text_index.json)
{
  "fields": [{
    "modality": "text",
    "model": "voyage-4",
    "path": "summary",
    "type": "autoEmbed"
  }]
}
```

Two parallel vector searches in the Memory Retrieval Agent

```
// Search 1: human-confirmed only (ground truth)
{ $vectorSearch: { index: "agent_memory_text_index", path: "summary",
  query: queryText, numCandidates: 50, limit: 10, model: "voyage-4" } },
{ $match: { source: "human_confirmed" } },
{ $limit: 6 }

// Search 2: all memories (including unverified agent conclusions)
{ $vectorSearch: { index: "agent_memory_text_index", path: "summary",
  query: queryText, numCandidates: 50, limit: 10, model: "voyage-4" } },
{ $limit: 6 }
```

The verified results are split into confirmed-fraud and confirmed-false-positive precedents. The unverified results provide additional context (prior agent reasoning).

Accuracy stat aggregation

```
{ $match: { type: "lesson_learned", source: "human_confirmed" } },
{ $match: { indicators: { $all: indicators } } },
{ $group: { _id: null, total: { $sum: 1 }, agreed: { $sum: { $cond: ["$agreement", 1, 0] } } } }
```

This runs as a standard MongoDB aggregation — not a vector search. It counts `lesson_learned` documents matching the current case's indicators and computes what percentage had `agreement`:

true.

The Compounding Effect

Each human review adds one `lesson_learned` document. The benefits compound across three dimensions:

1. Richer precedent briefs

With 5 confirmed-fraud precedents and 3 false-positive precedents, the investigation agent has a nuanced picture of when a pattern is real vs. benign. Contradictory lessons for the same indicators aren't a conflict — they're a **decision boundary**. The agent learns to distinguish which sub-pattern leads to which outcome.

2. More reliable accuracy stat

With 10+ data points per indicator combination, the stat is statistically meaningful. The Decision Agent's threshold-based policy (60% / 75%) becomes a real guardrail: patterns where the agent is frequently wrong get forced to human review, while patterns where it's consistently right can be auto-closed.

3. Semantic cross-pollination

`$vectorSearch` is semantic, not exact-match. A lesson about `[new_device]` from a user with 5 years of history might surface for a `[high_amount, new_device]` case if the summary text is semantically similar — even though the indicator arrays aren't identical. This means lessons from one pattern type can inform investigations of related patterns.

The Learning Loop in One Sentence

Every human review produces a vector-searchable lesson that becomes part of the precedent brief for the next investigation, and an accuracy stat that controls whether the Decision Agent trusts itself or escalates to a human — so the system gets smarter with every case and knows its own limits.